

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Xử lý tín hiệu số (CO2036)

Báo cáo Lab 3

Lớp L03 - HK252

Giảng viên hướng dẫn: Phạm Công Thái

Sinh viên thực hiện: Nguyễn Ngọc Quỳnh Nhi

Mã số sinh viên: 2212438

Thành phố Hồ Chí Minh, tháng 03 năm 2026

Mục lục

1	Giới Thiệu	2
2	Mục Tiêu	2
3	Phương Pháp	2
4	Nội dung thực hành	4
4.1	Exercise 1	4
4.2	Exercise 2	7
4.3	Exercise 3	10
4.4	Exercise 4	13
4.5	Exercise 5	18
4.6	Exercise 6	19
4.7	Exercise 2.1	23
4.8	Exercise 2.2	28
5	Bàn Luận	33
5.1	Kết quả đạt được	33
5.2	Những tồn tại và hạn chế	33
5.3	Định hướng phát triển kỹ năng	33
6	Tài liệu tham khảo	34

Tổng Quan

Báo cáo thực hành Lab 3 tập trung vào việc nghiên cứu và thực hiện các phép biến đổi cơ bản trên tín hiệu thời gian rời rạc. Nội dung chính bao gồm các thao tác dịch chuyển thời gian (shifting), gấp tín hiệu (folding), thay đổi thang thời gian (scaling) và phân tích các thành phần đối xứng (chẵn/lẻ) của tín hiệu. Toàn bộ các kết quả lý thuyết đều được kiểm chứng thông qua mô phỏng trên phần mềm Scilab.

1 Giới Thiệu

Trong lĩnh vực Xử lý tín hiệu số, việc hiểu rõ các phép biến đổi cơ bản là bước đầu tiên và quan trọng nhất để xử lý các hệ thống phức tạp hơn. Bài báo cáo này trình bày các phương pháp tác động lên biến số thời gian rời rạc n để quan sát sự thay đổi về mặt hình dáng và vị trí của tín hiệu trên trục thời gian. Đây là nền tảng cốt lõi cho các khái niệm như tổng chập (convolution) và các bộ lọc số sau này.

2 Mục Tiêu

Sau khi hoàn thành các bài thực hành trong Lab 3, các mục tiêu cụ thể đạt được bao gồm:

- **Về lý thuyết:** Nắm vững các quy tắc dịch (trễ/sớm), quy tắc gấp và thứ tự ưu tiên khi thực hiện nhiều phép biến đổi cùng lúc trên một tín hiệu.
- **Về kỹ năng:** Sử dụng thành thạo phần mềm Scilab để vẽ đồ thị tín hiệu rời rạc (lệnh `plot2d3`) và lập trình các thuật toán biến đổi chỉ số mảng.
- **Về phân tích:** Có khả năng tách một tín hiệu bất kỳ thành tổng của thành phần chẵn và thành phần lẻ.

3 Phương Pháp

Bài thực hành được thực hiện dựa trên sự kết hợp giữa lý thuyết toán học và công cụ mô phỏng:

- **Cơ sở lý thuyết:** Sử dụng các định nghĩa về tín hiệu đơn vị $\delta(n)$, hàm nấc $u(n)$ và các công thức biến đổi biến số độc lập $x(an + b)$.

-
- **Công cụ phần mềm:** Sử dụng môi trường lập trình **Scilab 2026** để hiện thực hóa các biểu thức toán học.
 - **Quy trình thực hiện:** Tính toán giá trị biên độ tại từng mẫu thời gian n bằng tay, sau đó viết mã nguồn (script) để tự động hóa việc vẽ đồ thị và so sánh kết quả.

4 Nội dung thực hành

4.1 Exercise 1

Xây dựng hàm `[yn, yorigin] = delay(xn, xorigin, k)` để thực hiện phép toán làm trễ (**delay**) tín hiệu $y(n) = x(n - k)$ với $k > 0$. Trong đó:

- **xn**: Vector biểu diễn tín hiệu rời rạc đầu vào $x(n)$.
- **xorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu $x(n)$.
- **yn**: Vector biểu diễn tín hiệu đầu ra sau khi làm trễ.
- **yorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu đầu ra $y(n)$.
- **Yêu cầu**: Cả hai tín hiệu $x(n)$ và $y(n)$ phải được hiển thị trên cùng một hình vẽ.

Ví dụ minh họa: Để xác định tín hiệu trễ $y(n) = x(n - 1)$ với $x(n) = \{1, -2, 3, 6\}$, hàm sẽ được gọi như sau:

```
[yn, yorigin] = delay([1, -2, 3, 6], 3, 1)
```

Kết quả mong đợi:

- `yn = [1, -2, 3, 6]`
- `yorigin = 2`.

Bài làm

- Xác định tín hiệu trễ $y(n)$ của tín hiệu $x(n)$:

Phép **delay** tín hiệu không làm thay đổi giá trị biên độ tín hiệu mà chỉ dịch tín hiệu sang phải k đơn vị trên trục thời gian. Điều này cũng có nghĩa là hệ trục tọa độ sẽ dịch sang trái k đơn vị $\Leftrightarrow$ gốc tọa độ của tín hiệu cũng dịch sang trái k đơn vị. Do đó `yn = xn` và `yorigin = xorigin - k`.

- Vẽ tín hiệu gốc $x(n)$ và tín hiệu trễ $y(n)$ của tín hiệu $x(n)$:

Để vẽ được sơ đồ biểu diễn tín hiệu, ta cần xác định được trục thời gian n của tín hiệu. Tức là, chúng ta cần ánh xạ giữa chỉ số mảng (index) (luôn bắt đầu từ 1 trong Scilab) và biến thời gian rời rạc n bằng công thức sau: $n = index - origin_index$;

Đoạn code Scilab thực hiện yêu cầu Exercise 1

```
1 clear(); clc; clf();
2
3 function [yn, yorigin] = delay(xn, xorigin, k)
4     yn = xn;
5     yorigin = xorigin - k;
6 endfunction
7
8 xn = [1, -2, 3, 6];
9 xorigin = 3;
10 k = 1;
11 [yn, yorigin] = delay(xn, xorigin, k);
12 disp(xn);
13 disp(xorigin);
14 disp(yn);
15 disp(yorigin);
16
17 // --- Dynamic Calculation the values of n-axis ---
18 // n = index - origin_index
19 n_x = (1:size(xn, '*')) - xorigin;
20 n_y = (1:size(yn, '*')) - yorigin;
21
22 subplot(2, 1, 1);
23 plot2d3(n_x, xn);
24 plot(n_x, xn, "ro");
25 title("Original Signal x(n)");
26 xlabel("n"); ylabel("Aplitude");
27 xgrid(color('gray'));
28
29 subplot(2, 1, 2);
30 plot2d3(n_y, yn);
31 plot(n_y, yn, "ro");
32 title("y(n) = x(n - 1)");
33 xlabel("n"); ylabel("Aplitude");
34 xgrid(color('gray'));
35
36
```

```
Scilab 2026.0.1 Console

-->

--> function [yn, yorigin] = delay(xn, xorigin, k)
>     yn = xn;
>     yorigin = xorigin - k;
> endfunction

-->

--> xn = [1, -2, 3, 6];

--> xorigin = 3;

--> k = 1;

--> [yn, yorigin] = delay(xn, xorigin, k);

--> disp(xn);

1. -2. 3. 6.

--> disp(xorigin);

3.

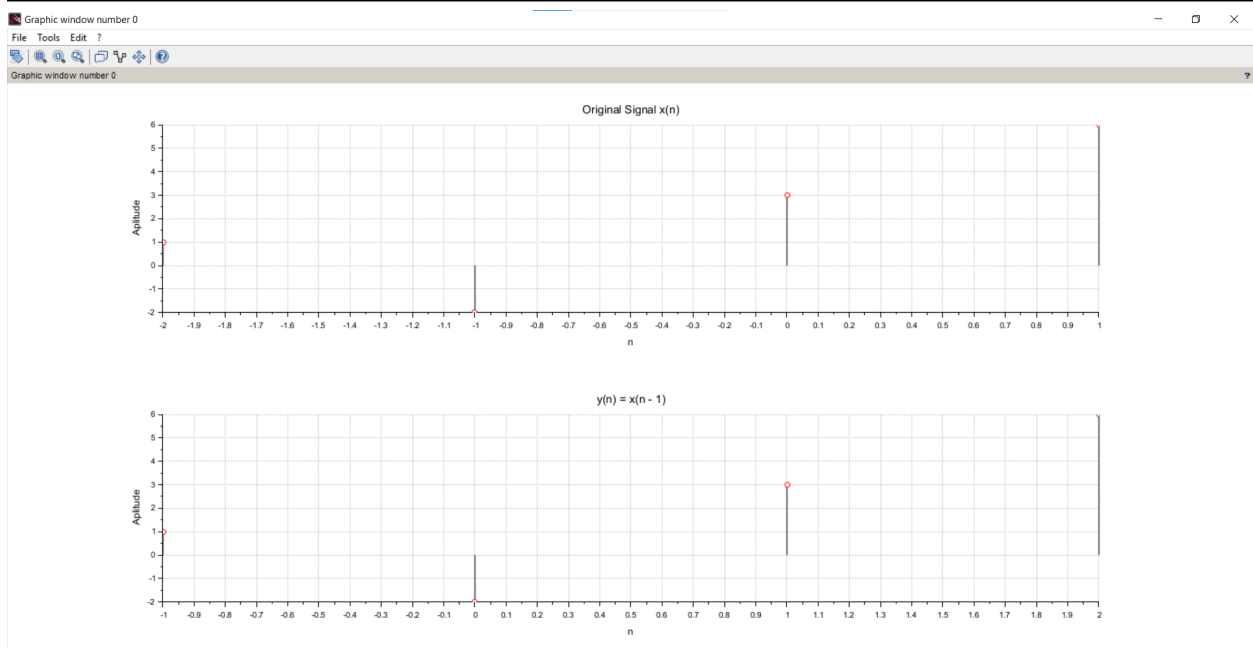
--> disp(yn);

1. -2. 3. 6.

--> disp(yorigin);

2.
```

Hình 1: Kết quả tín hiệu $y(n) = x(n - 1)$.



Hình 2: Hình ảnh đồ thị tín hiệu $x(n)$ và tín hiệu $y(n) = x(n - 1)$.

4.2 Exercise 2

Xây dựng hàm `[yn, yorigin] = advance(xn, xorigin, k)` để thực hiện phép toán làm sớm (*advance*) tín hiệu $y(n) = x(n + k)$ với $k > 0$. Trong đó:

- **xn**: Vector biểu diễn tín hiệu rời rạc đầu vào $x(n)$.
- **xorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu $x(n)$.
- **yn**: Vector biểu diễn tín hiệu đầu ra sau khi làm sớm *advance* tín hiệu.
- **yorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu đầu ra $y(n)$.
- Yêu cầu: Cả hai tín hiệu $x(n)$ và $y(n)$ phải được hiển thị trên cùng một hình vẽ.

Bài làm

- Xác định tín hiệu sớm (*advanced*) $y(n)$ của tín hiệu $x(n)$:

Phép *advance* tín hiệu không làm thay đổi giá trị biên độ tín hiệu mà chỉ dịch tín hiệu sang trái k đơn vị trên trục thời gian. Điều này cũng có nghĩa là hệ trục tọa độ sẽ dịch sang phải k đơn vị $\Leftrightarrow$ gốc tọa độ của tín hiệu cũng dịch sang phải k đơn vị. Do đó $y_n = x_n$ và $yorigin = xorigin + k$.

- Vẽ tín hiệu gốc $x(n)$ và tín hiệu sớm (*advanced*) $y(n)$ của tín hiệu $x(n)$:

Để vẽ được sơ đồ biểu diễn tín hiệu, ta cần xác định được trục thời gian n của tín hiệu. Tức là, chúng ta cần ánh xạ giữa chỉ số mảng (index) (luôn bắt đầu từ 1 trong Scilab) và biến thời gian rời rạc n bằng công thức sau: $n = index - origin_index$;

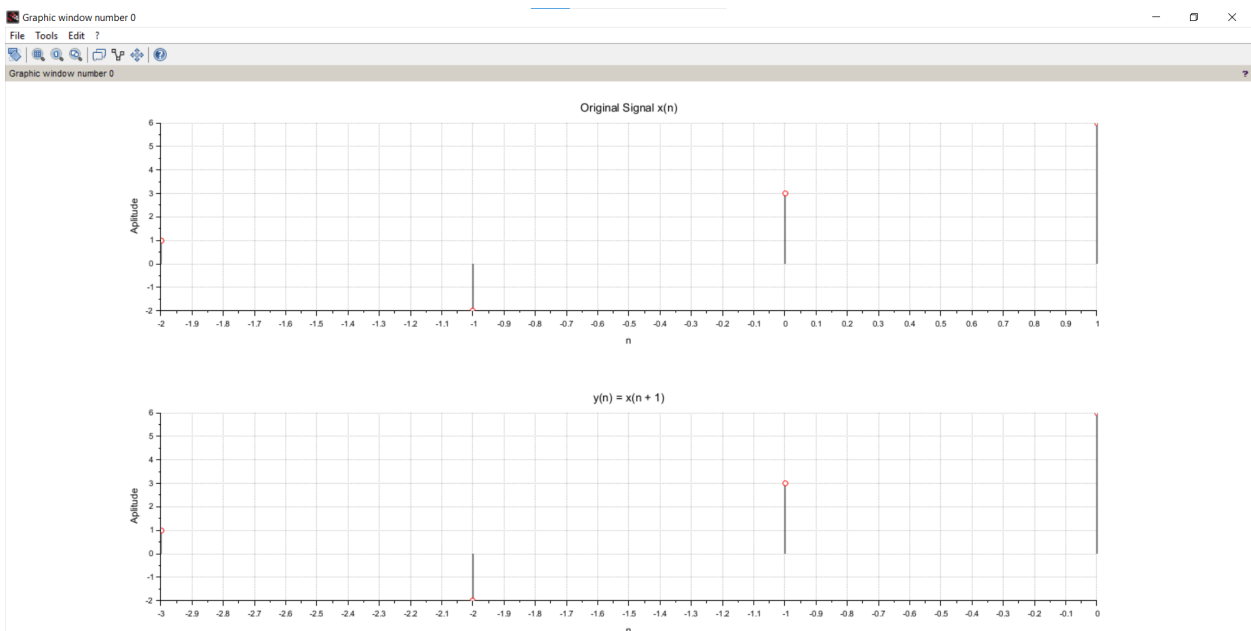
Đoạn code Scilab thực hiện yêu cầu Exercise 2

```
1 clear(); clc; clf();
2
3 function [yn, yorigin] = advance(xn, xorigin, k)
4     yn = xn;
5     yorigin = xorigin + k;
6 endfunction
7
8 xn = [1, -2, 3, 6];
9 xorigin = 3;
10 k = 1;
11 [yn, yorigin] = advance(xn, xorigin, k);
12 disp(xn);
13 disp(xorigin);
14 disp(yn);
15 disp(yorigin);
16
17 // --- Dynamic Calculation the values of n-axis ---
18 // n = index - origin_index
19 n_x = (1:size(xn, '*')) - xorigin;
20 n_y = (1:size(yn, '*')) - yorigin;
21
22 subplot(2, 1, 1);
23 plot2d3(n_x, xn);
24 plot(n_x, xn, "ro");
25 title("Original Signal x(n)");
26 xlabel("n"); ylabel("Aplitude");
27 xgrid(color('gray'));
28
29 subplot(2, 1, 2);
30 plot2d3(n_y, yn);
31 plot(n_y, yn, "ro");
32 title("y(n) = x(n + 1)");
```

```
33 xlabel("n"); ylabel("Aplitude");
34 xgrid(color('gray'));
35
36
```

```
Scilab 2026 0.1 Console
--> function [yn, yorigin] = advance(xn, xorigin, k)
-->     yn = xn;
-->     yorigin = xorigin + k;
--> endfunction
--> xn = [1, -2, 3, 6];
--> xorigin = 3;
--> k = 1;
--> [yn, yorigin] = advance(xn, xorigin, k);
--> disp(xn);
-->     1.  -2.   3.   6.
--> disp(xorigin);
-->     3.
--> disp(yn);
-->     1.  -2.   3.   6.
--> disp(yorigin);
-->     4.
```

Hình 3: Kết quả tín hiệu $y(n) = x(n + 1)$.



Hình 4: Hình ảnh đồ thị tín hiệu $x(n)$ và tín hiệu $y(n) = x(n + 1)$.

4.3 Exercise 3

Xây dựng hàm `[yn, yorigin] = fold(xn, xorigin, k)` để thực hiện phép toán làm đảo (**fold**) tín hiệu $y(n) = x(-n)$ với. Trong đó:

- **xn**: Vector biểu diễn tín hiệu rời rạc đầu vào $x(n)$.
- **xorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu $x(n)$.
- **yn**: Vector biểu diễn tín hiệu đầu ra sau khi thực hiện đảo (**fold**) tín hiệu.
- **yorigin**: Vị trí gốc tọa độ ($n = 0$) của tín hiệu đầu ra $y(n)$.
- Yêu cầu: Cả hai tín hiệu $x(n)$ và $y(n)$ phải được hiển thị trên cùng một hình vẽ.

Bài làm

- Xác định tín hiệu đảo (**folded**) $y(n) = x(-n)$ của tín hiệu $x(n)$:

Sau khi thực hiện phép đảo tín hiệu, các giá trị biên độ của tín hiệu bị lấy đối xứng qua trục tung so với tín hiệu gốc ban đầu. Tức là, gốc tọa độ của tín hiệu vẫn giữ nguyên và giá trị biên độ của tín hiệu tại gốc tọa độ vẫn không thay đổi. Tuy nhiên, vì chuỗi giá trị của tín hiệu đang được lưu ở dạng vector, và trong Scilab, chỉ số của vector bắt đầu từ 1, nên giá trị **yorigin** sẽ khác với **xorigin**.

Ví dụ: Cho tín hiệu $x(n) = \{1, 2, 5\uparrow, 7, -3, -4\}$ thì $xorigin = 3$. Tín hiệu đảo của $x(n)$ là $y(n) = x(-n) = \{-4, -3, 7, 5\uparrow, 2, 1\}$ thì $yorigin = 4$.

Do đó ta cần tính chỉ số tại gốc tọa độ của tín hiệu sau khi đảo bằng công thức:

$$yorigin = length_x - xorigin \quad (1)$$

Trong đó:

- $length_x$ là số phần tử của vector **xn**.
- **yorigin** là chỉ số tại gốc tọa độ của tín hiệu **yn**.
- **xorigin** là chỉ số tại gốc tọa độ của tín hiệu **xn**.
- Vẽ tín hiệu gốc $x(n)$ và tín hiệu đảo (**folded**) $y(n)$ của tín hiệu $x(n)$:

Để vẽ được sơ đồ biểu diễn tín hiệu, ta cần xác định được trục thời gian n của tín hiệu. Tức là, chúng ta cần ánh xạ giữa chỉ số mảng (index) (luôn bắt đầu từ 1 trong Scilab) và biến thời gian rời rạc n bằng công thức sau: $n = index - origin_index$;

Đoạn code Scilab thực hiện yêu cầu Exercise 3

```
1 clear();    clc;    clf();
2
3 function [yn, yorigin] = fold(xn, xorigin)
4     length_x = length(xn);
5     yorigin = length_x - xorigin + 1;
6
7     for i = 0 : 5
8         yn(1, i + 1) = xn(length_x - i);
9     end
10
11 endfunction
12
13 xn = [1, 2, 5, 7, -3, -4];
14 xorigin = 3;
15 [yn, yorigin] = fold(xn, xorigin);
16 disp(xn);
17 disp(xorigin);
18 disp(yn);
19 disp(yorigin);
20
21 n_x = (1 : size(xn, '*')) - xorigin;
22 n_y = (1 : size(yn, '*')) - yorigin;
23
24 subplot(2, 1, 1);
25 plot2d3(n_x, xn);
26 plot(n_x, xn, "ro");
27 title("Original signal x(n)");
28 xlabel("n"); ylabel("Aplitude");
29 xgrid(color('gray'));
30
31 subplot(2, 1, 2);
32 plot2d3(n_y, yn);
33 plot(n_y, yn, "ro");
34 title("Folded signal y(n) = x(-n)");
35 xlabel("n"); ylabel("Aplitude");
36 xgrid(color('gray'));
37
```

```
Scilab 2026.0.1 Console

-->

--> function [yn, yorigin] = fold(xn, xorigin)
>     length_x = length(xn);
>     yorigin = length_x - xorigin + 1;
>
>     for i = 0 : 5
>         yn(1, i + 1) = xn(length_x - i);
>     end
>
> endfunction

-->

--> xn = [1, 2, 5, 7, -3, -4];

--> xorigin = 3;

--> [yn, yorigin] = fold(xn, xorigin);

--> disp(xn);

    1.    2.    5.    7.   -3.   -4.

--> disp(xorigin);

    3.

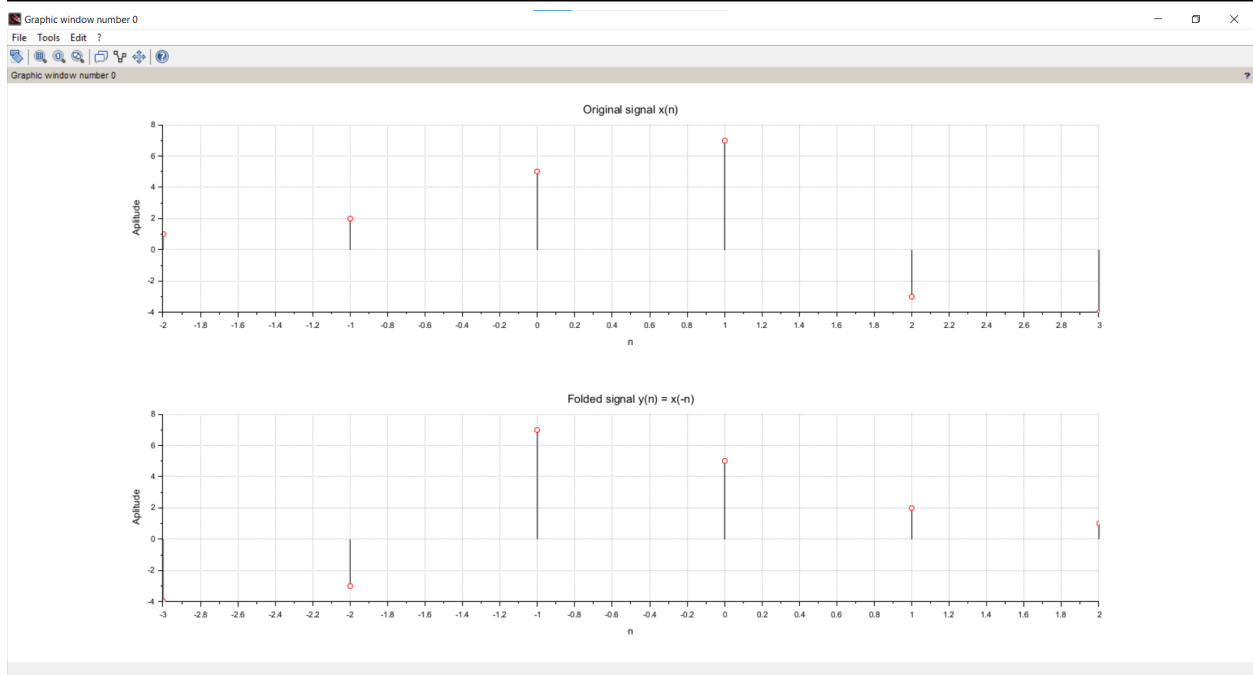
--> disp(yn);

   -4.   -3.    7.    5.    2.    1.

--> disp(yorigin);

    4.
```

Hình 5: Kết quả tín hiệu $y(n) = x(-n)$



Hình 6: Hình ảnh đồ thị tín hiệu gốc $x(n)$ và tín hiệu đảo $y(n) = x(-n)$

4.4 Exercise 4

Xây dựng hàm `[yn, yorigin] = add(x1n, x1origin, x2n, x2origin)` thực hiện phép cộng tín hiệu $y(n) = x_1(n) + x_2(n)$, trong đó:

- Các tín hiệu thời gian rời rạc $x_1(n)$ và $x_2(n)$ được biểu diễn bằng các vector `x1n` và `x2n` tương ứng.
- `x1origin` và `x2origin` chỉ vị trí gốc tọa độ (vị trí của $n = 0$) của các tín hiệu $x_1(n)$ và $x_2(n)$.
- `yn` là vector của tín hiệu ngõ ra.
- `yorigin` chỉ vị trí gốc tọa độ của tín hiệu $y(n)$.
- $x_1(n)$, $x_2(n)$ và $y(n)$ được hiển thị dưới dạng đồ thị rời rạc trong cùng một hình vẽ.

Ví dụ: Để xác định tín hiệu tổng $y(n) = x_1(n) + x_2(n)$ với $x_1(n) = \{0 \uparrow, 1, 3, -2\}$ và $x_2(n) = \{1, 1 \uparrow, 2, 3\}$, bạn có thể gọi hàm như sau:

```
[yn, yorigin] = add([0, 1, 3, -2], 1, [1, 1, 2, 3], 2)
```

Kết quả đầu ra sẽ là:

- $y_n = [1, 1, 3, 6, -2]$
- $y_{origin} = 1$

Ngoài ra, bạn phải hiển thị các tín hiệu $x_1(n)$, $x_2(n)$ và $y(n)$ bằng đồ thị trong cùng một cửa sổ để so sánh.

Bài làm

• Xây dựng hàm $[y_n, y_{origin}] = \text{add}(x1_n, x1_{origin}, x2_n, x2_{origin})$ thực hiện phép cộng tín hiệu $y(n) = x_1(n) + x_2(n)$:

Để thực hiện cộng 2 tín hiệu $x1(n)$ và $x2(n)$ ta cần thêm các giá trị 0 vào tín hiệu để đảm bảo 2 vector $x1(n)$ và $x2(n)$ có kích thước bằng nhau.

Đoạn code Scilab thực hiện thêm giá trị 0 vào vecto

```
1 function a = zeroFront(xn, gap)
2     for i = 1 : gap
3         a(1, i) = 0;
4     end
5
6     aLength = length(a);
7     xnLength = length(xn);
8     for i = 1 : xnLength
9         a(1, aLength + i) = xn(i);
10    end
11 endfunction
12
13 function b = zeroBack(xn, denta)
14     xnLength = length(xn);
15     for i = 1 : denta
16         xn(1, xnLength + i) = 0;
17     end
18     b = xn;
19 endfunction
20
```

Hiện thực hàm cộng 2 tín hiệu:

- Đầu tiên, cần thêm các giá trị 0 vào các vị trí bên trái gốc tọa độ của tín hiệu có vị trí gốc tọa độ nhỏ hơn để đảm bảo 2 gốc tọa độ của 2 tín hiệu có vị trí tương ứng.
- Sau đó, cần thêm các giá trị 0 vào các vị trí bên phải gốc tọa độ của tín hiệu có số phần tử ít hơn để đảm bảo 2 tín hiệu có kích thước bằng nhau để thực hiện được phép cộng 2 vecto.
- Dùng toán tử + để thực hiện cộng 2 tín hiệu rồi gán kết quả vào biến $y_n = x_{1n} + x_{2n}$;
- Tín hiệu kết quả sẽ có chỉ số gốc tọa độ (vị trí của gốc tọa độ trong vecto) là chỉ số gốc tọa độ lớn hơn trong 2 tín hiệu x_{1n} và x_{2n} .

Đoạn code Scilab hiện thực hàm cộng 2 tín hiệu

```
1 function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
2     gap = abs(x1origin - x2origin);
3
4     if x1origin < x2origin then
5         yorigin = x2origin;
6
7         x1n = zeroFront(x1n, gap);
8
9         denta = abs(length(x1n) - length(x2n));
10        if (length(x1n) > length(x2n)) then
11            x2n = zeroBack(x2n, denta);
12        elseif length(x1n) < length(x2n) then
13            x1n = zeroBack(x1n, denta);
14        end
15    elseif x2origin < x1origin then
16        yorigin = x1origin;
17
18        x2n = zeroFront(x2n, gap);
19
20        denta = abs(length(x2n) - length(x1n));
21        if (length(x2n) > length(x1n)) then
22            x1n = zeroBack(x1n, denta);
23        elseif length(x2n) < length(x1n) then
24            x2n = zeroBack(x2n, denta);
25        end
26    else
```



```

27     yorigin = x1origin;
28     denta = length(x1n) - length(x2n);
29     if (denta > 0) then
30         x2n = zeroBack(x2n, denta);
31     elseif denta < 0 then
32         x1n = zeroBack(x1n, -denta);
33     end
34 end
35
36 yn = x1n + x2n;
37
38 endfunction
39

```

- Vẽ sơ đồ 3 tín hiệu $x1(n)$, $x2(n)$ và $y(n)$ trên cùng 1 cửa sổ đồ họa:

Đoạn code Scilab vẽ sơ đồ 3 tín hiệu

```

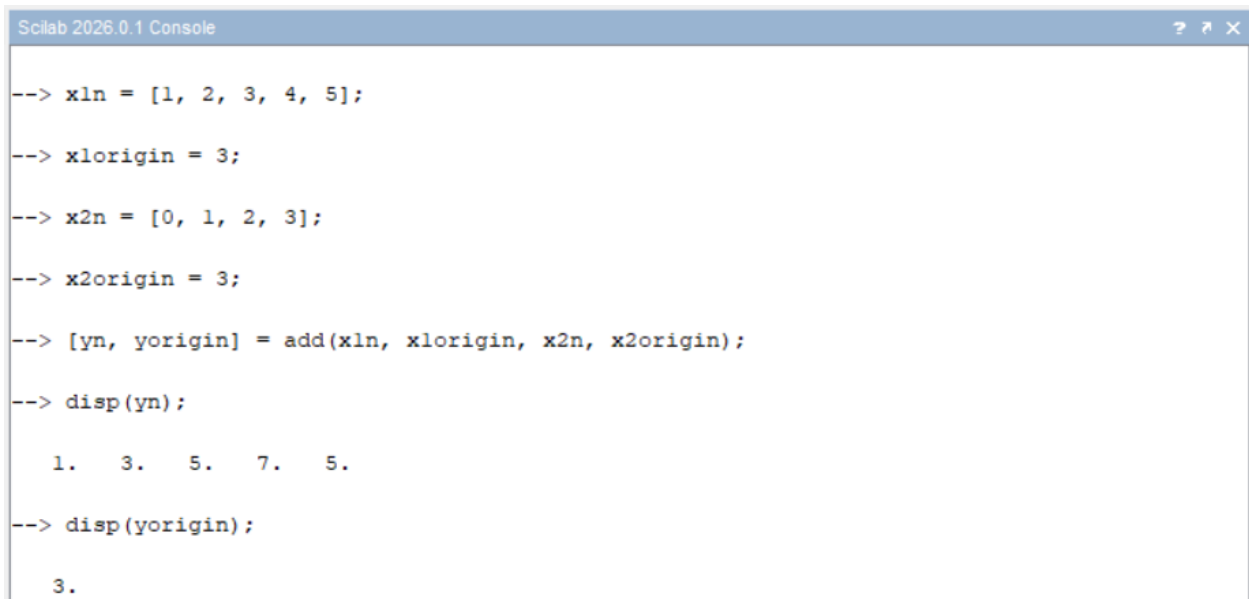
1 x1n = [1, 2, 3, 4, 5];
2 x1origin = 3;
3 x2n = [0, 1, 2, 3];
4 x2origin = 3;
5 [yn, yorigin] = add(x1n, x1origin, x2n, x2origin);
6 disp(yn);
7 disp(yorigin);
8
9 n_x1 = (1 : size(x1n, '*')) - x1origin;
10 n_x2 = (1 : size(x2n, '*')) - x2origin;
11 n_y = (1 : size(yn, '*')) - yorigin;
12
13 timeMin = min([min(n_x1), min(n_x2), min(n_y)]);
14 timeMax = max([max(n_x1), max(n_x2), max(n_y)]);
15 timeLimit = [timeMin, timeMax];
16 Amin = min([min(x1n), min(x2n), min(yn)]);
17 Amax = max([max(x1n), max(x2n), max(yn)]);
18 ALimit = [Amin, Amax];
19
20 subplot(3, 1, 1);
21 plot2d3(n_x1, x1n);
22 plot(n_x1, x1n, "ro");

```

```

23 gca().data_bounds = [timeMin - 1, Amin - 1; timeMax + 1, Amax + 1];
24 title("Original signal x1(n)");
25 xlabel("n"); ylabel("Aplitude");
26 xgrid(color('gray'));
27
28 subplot(3, 1, 2);
29 plot2d3(n_x2, x2n);
30 plot(n_x2, x2n, "ro");
31 gca().data_bounds = [timeMin - 1, Amin - 1; timeMax + 1, Amax + 1];
32 title("Original signal x2(n)");
33 xlabel("n"); ylabel("Aplitude");
34 xgrid(color('gray'));
35
36 subplot(3, 1, 3);
37 plot2d3(n_y, yn);
38 plot(n_y, yn, "ro");
39 gca().data_bounds = [timeMin - 1, Amin - 1; timeMax + 1, Amax + 1];
40 title("Sum signal y(n) = x1(n) + x2(n)");
41 xlabel("n"); ylabel("Aplitude");
42 xgrid(color('gray'));
43

```

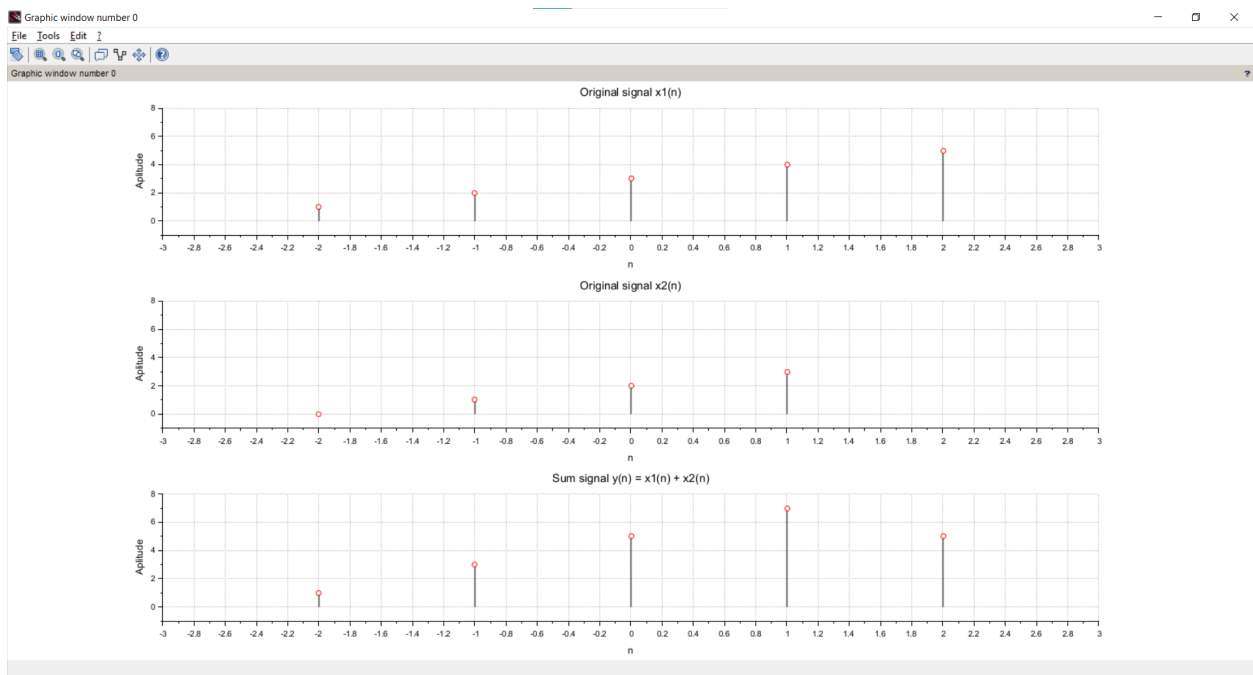


```

Scilab 2026.0.1 Console
--> xln = [1, 2, 3, 4, 5];
--> xlorigin = 3;
--> x2n = [0, 1, 2, 3];
--> x2origin = 3;
--> [yn, yorigin] = add(xln, xlorigin, x2n, x2origin);
--> disp(yn);
    1.   3.   5.   7.   5.
--> disp(yorigin);
    3.

```

Hình 7: Kết quả đoạn code Scilab thực hiện Exercise 4.



Hình 8: Hình ảnh sơ đồ tín hiệu gốc $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n) = x_1(n) + x_2(n)$

4.5 Exercise 5

Xây dựng hàm `[yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)` thực hiện phép nhân tín hiệu $y(n) = x_1(n) \cdot x_2(n)$, trong đó:

- Các tín hiệu thời gian rời rạc $x_1(n)$ và $x_2(n)$ được biểu diễn bởi các vector `x1n` và `x2n` tương ứng.
- `x1origin` và `x2origin` chỉ vị trí gốc tọa độ của các tín hiệu $x_1(n)$ và $x_2(n)$ tương ứng.
- `yn` là vector của tín hiệu ngõ ra.
- `yorigin` chỉ vị trí gốc tọa độ của tín hiệu $y(n)$.
- $x_1(n)$, $x_2(n)$ và $y(n)$ được hiển thị bằng đồ thị trong cùng một hình.

Bài làm

Thực hiện tương tự Exercise 4 nhưng dùng toán tử `.*` để thực hiện nhân tương ứng các phần tử của 2 vecto: `yn = x1n .* x2n;`

```
Scilab 2026.0.1 Console

--> x1n = [1, 2, 3, 4, 5];

--> x1origin = 3;

--> x2n = [0, 1, 2, 3];

--> x2origin = 3;

--> [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin);

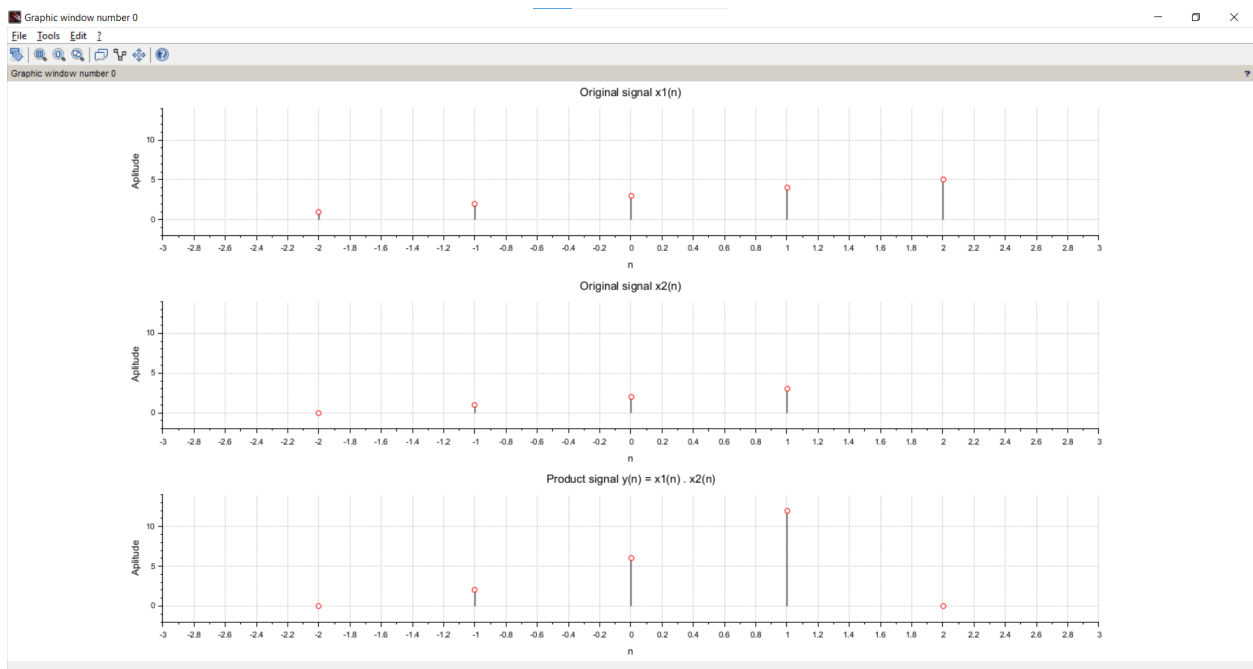
--> disp(yn);

    0.   2.   6.  12.   0.

--> disp(yorigin);

    3.
```

Hình 9: Kết quả đoạn code Scilab thực hiện yêu cầu Exercise 5.



Hình 10: Hình ảnh đồ thị tín hiệu gốc $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n) = x_1(n).x_2(n)$

4.6 Exercise 6

Xây dựng hàm `[yn, yorigin] = convolution(xn, xorigin, hn, horigin)` thực hiện phép toán chập $y(n) = x(n) * h(n)$, trong đó:

- xn là tín hiệu đầu vào và hn là hàm đặc tính (đáp ứng xung) của hệ thống.

- x_{origin} và h_{origin} lần lượt chỉ vị trí gốc tọa độ ($n = 0$) của tín hiệu $x(n)$ và $h(n)$.
- y_n là vector của tín hiệu đầu ra.
- y_{origin} chỉ vị trí gốc tọa độ của tín hiệu $y(n)$.

Bài làm

Thuật toán thực hiện phép chập $y(n) = x(n) * h(n)$ được chia thành các bước chính sau:

1. **Xác định phạm vi thời gian của tín hiệu ngõ ra:** Dựa trên lý thuyết về phép chập, phạm vi thời gian của $y(n)$ được xác định bởi:

$$n_{start,y} = n_{start,x} + n_{start,h}$$

$$n_{end,y} = n_{end,x} + n_{end,h}$$

Từ đó, độ dài của vector kết quả là $L_y = L_x + L_h - 1$.

2. **Xác định gốc tọa độ cho tín hiệu ngõ ra:** Vị trí gốc tọa độ của $y(n)$ được tính toán dựa trên điểm bắt đầu $n_{start,y}$ để ánh xạ chính xác vào chỉ số mảng của Scilab:

$$y_{origin} = 1 - n_{start,y}$$

3. **Tính toán giá trị biên độ bằng vòng lặp lồng nhau:** Sử dụng hai vòng lặp để thực hiện tổng chập rời rạc:

- Vòng lặp ngoài chạy theo biến thời gian n của tín hiệu ngõ ra từ $n_{start,y}$ đến $n_{end,y}$.
- Vòng lặp trong chạy theo biến thời gian k của tín hiệu đầu vào $x(n)$.
- Tại mỗi bước, tính toán các chỉ số mảng tương ứng: $i_x = k + x_{origin}$ và $i_h = (n - k) + h_{origin}$.
- **Kiểm tra điều kiện biên:** Chỉ thực hiện phép nhân $x(i_x) \cdot h(i_h)$ và cộng dồn vào $y(n)$ khi cả hai chỉ số i_x và i_h đều nằm trong phạm vi hợp lệ của các vector đầu vào.

Đoạn code Scilab hiện thực hàm convolution

```
1 function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
2     Lx = length(xn);
3     Lh = length(hn);
4     n_start_x = 0 - xorigin + 1;
5     n_end_x = n_start_x + Lx - 1;
6     n_start_h = 0 - horigin + 1;
7     n_end_h = n_start_h + Lh - 1;
8     n_start_y = n_start_x + n_start_h;
9     n_end_y = n_end_x + n_end_h;
10    yorigin = 1 - n_start_y;
11
12    for n = n_start_y : n_end_y
13        i_y = n + yorigin;
14        yn(1, i_y) = 0;
15        for k = n_start_x : n_end_x
16            i_x = k + xorigin;
17            i_h = (n - k) + horigin;
18            if (i_x >= 1 && i_x <= Lx) && (i_h >= 1 && i_h <= Lh) then
19                yn(1, i_y) = yn(1, i_y) + xn(i_x) * hn(i_h);
20            end
21        end
22    end
23
24 endfunction
25
```

```
Scilab 2026.0.1 Console

--> xn = [1, 2, 3, 1];

--> xorigin = 1;

--> hn = [1, 2, 1, -1];

--> horigin = 2;

--> [yn, yorigin] = convolution(xn, xorigin, hn, horigin);

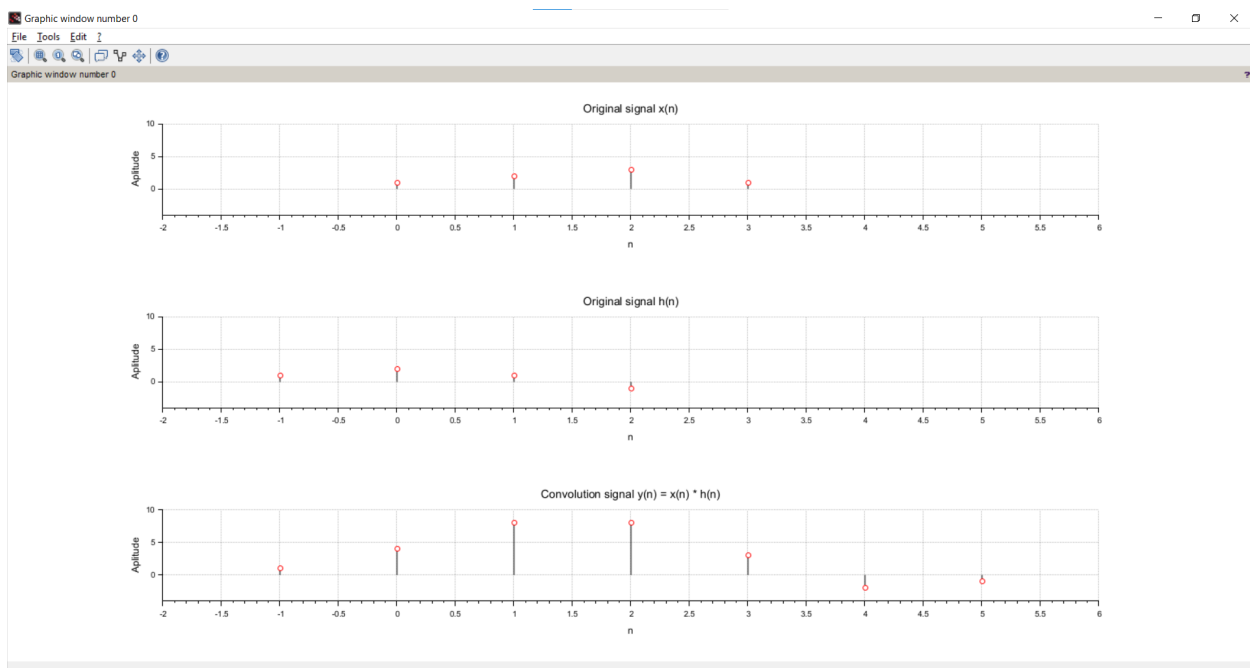
--> disp(yn);

    1.    4.    8.    8.    3.   -2.   -1.

--> disp(yorigin);

    2.
```

Hình 11: Kết quả đoạn code Scilab hiện thực hàm `convolution` theo yêu cầu Exercise 6.



Hình 12: Hình ảnh đồ thị tín hiệu $x(n)$, $h(n)$ và tín hiệu tích chập $y(n) = x(n) * h(n)$

4.7 Exercise 2.1

Một tín hiệu thời gian rời rạc $x(n)$ được định nghĩa như sau:

$$x(n) = \begin{cases} 1 + \frac{n}{3}, & -3 \leq n \leq -1 \\ 1, & 0 \leq n \leq 3 \\ 0, & \text{ở các nơi khác} \end{cases}$$

- (a) Xác định các giá trị và vẽ đồ thị tín hiệu $x(n)$.
- (b) Vẽ đồ thị các tín hiệu thu được nếu chúng ta:
 - 1. Đầu tiên lật $x(n)$ và sau đó làm trễ tín hiệu kết quả đi bốn mẫu.
 - 2. Đầu tiên làm trễ $x(n)$ đi bốn mẫu và sau đó lật tín hiệu kết quả.
- (c) Vẽ đồ thị tín hiệu $x(-n+4)$.
- (d) So sánh các kết quả ở phần (b) và (c), từ đó rút ra quy tắc để thu được tín hiệu $x(-n+k)$ từ $x(n)$.
- (e) Bạn có thể biểu diễn tín hiệu $x(n)$ theo các tín hiệu xung đơn vị $\delta(n)$ và bước đơn vị $u(n)$ không?

Bài làm

- (a) Xác định các giá trị và vẽ đồ thị tín hiệu $x(n)$.

Đoạn code Scilab xác định và vẽ sơ đồ tín hiệu $x(n)$

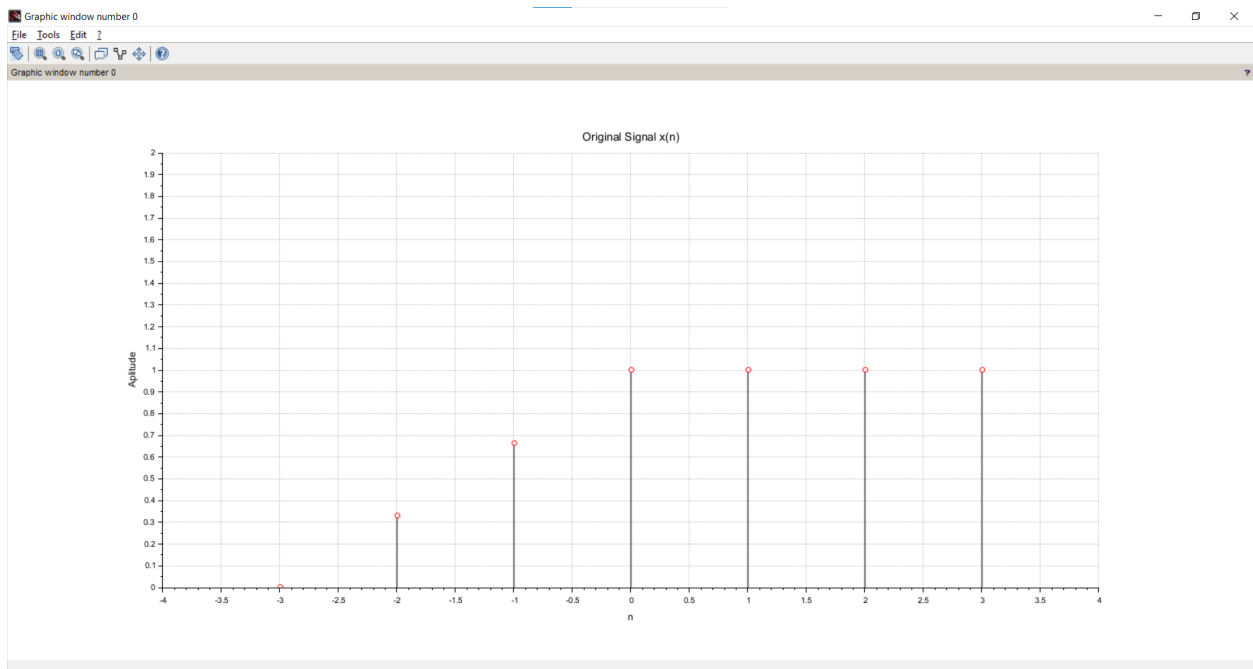
```
1 clear();    clc;    clf();
2
3 n_start = -3;
4 n_end = 3;
5 xorigin = 0 - n_start + 1;
6
7 for n = n_start : n_end
8     i_x = n + xorigin;
9     if (n >= -3 && n <= -1) then
10         xn(1, i_x) = 1 + n/3;
```



```

11     elseif (n >= 0 && n <= 3) then
12         xn(1, i_x) = 1;
13     else
14         xn(1, i_x) = 0;
15     end
16 end
17
18 disp(xn);
19
20
21 n_x = (1:size(xn, '*')) - xorigin;
22
23 plot2d3(n_x, xn);
24 plot(n_x, xn, "ro");
25 gca().data_bounds = [n_start - 1, min(xn); n_end + 1, max(xn) + 1];
26 title("Original Signal x(n)");
27 xlabel("n"); ylabel("Aplitude");
28 xgrid(color('gray'));
29

```



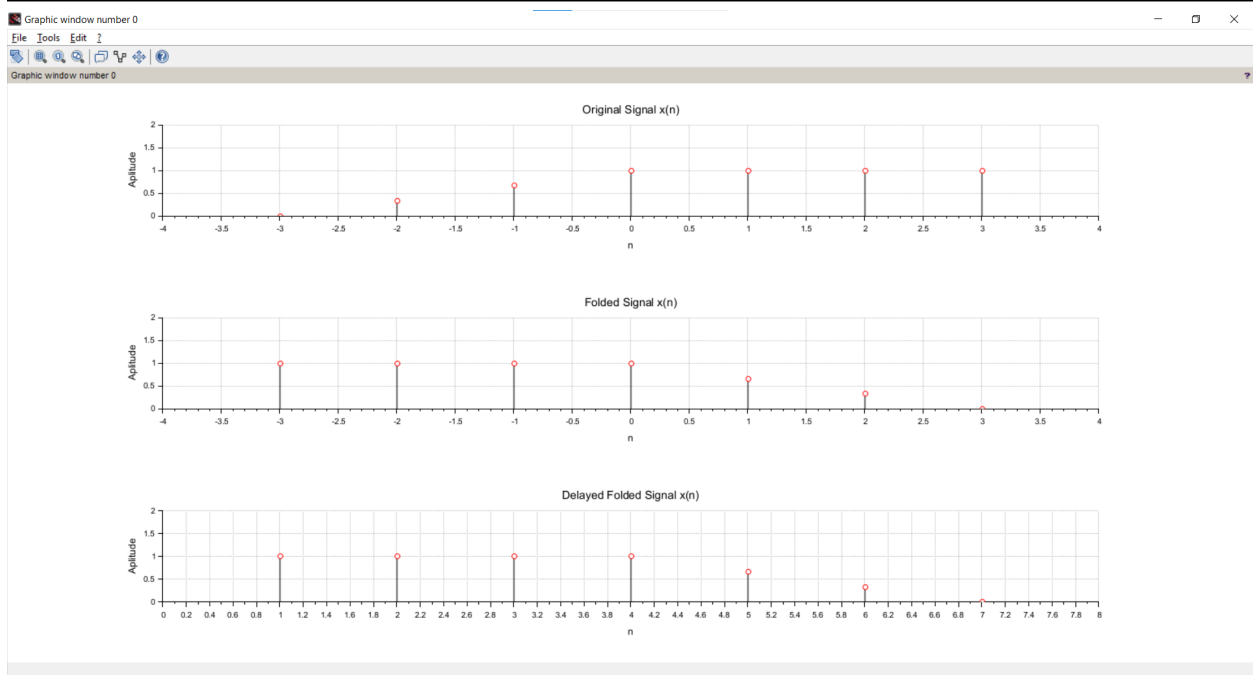
Hình 13: Sơ đồ tín hiệu $x(n)$

(b) Vẽ đồ thị các tín hiệu thu được nếu chúng ta:

1. Đầu tiên lật $x(n)$ và sau đó làm trễ tín hiệu kết quả đi bốn mẫu.

Đoạn code Scilab lật $x(n)$ rồi tiếp tục làm trễ tín hiệu đi 4 mẫu rồi vẽ tín hiệu kết quả.

```
1 //----- Exercise 2.1b1 -----
2 function [yn, yorigin] = fold(xn, xorigin)
3     length_x = length(xn);
4     yorigin = length_x - xorigin + 1;
5     for i = 0 : (length_x - 1)
6         yn(1, i + 1) = xn(length_x - i);
7     end
8 endfunction
9
10 function [yn, yorigin] = delay(xn, xorigin, k)
11     yn = xn;
12     yorigin = xorigin - k;
13 endfunction
14
15 disp(xn);
16 subplot(3, 1, 1);
17 draw(xn, xorigin);
18 title("Original Signal x(n)");
19
20 [xn_fold, xorigin_fold] = fold(xn, xorigin);
21 disp(xn_fold);
22 subplot(3, 1, 2);
23 draw(xn_fold, xorigin_fold);
24 title("Folded Signal x(n)");
25
26 [xnb1, xnb1origin] = delay(xn_fold, xorigin_fold, 4);
27 disp(xnb1);
28 disp(xnb1origin);
29 subplot(3, 1, 3);
30 draw(xnb1, xnb1origin);
31 title("Delayed Folded Signal x(n)");
32
```

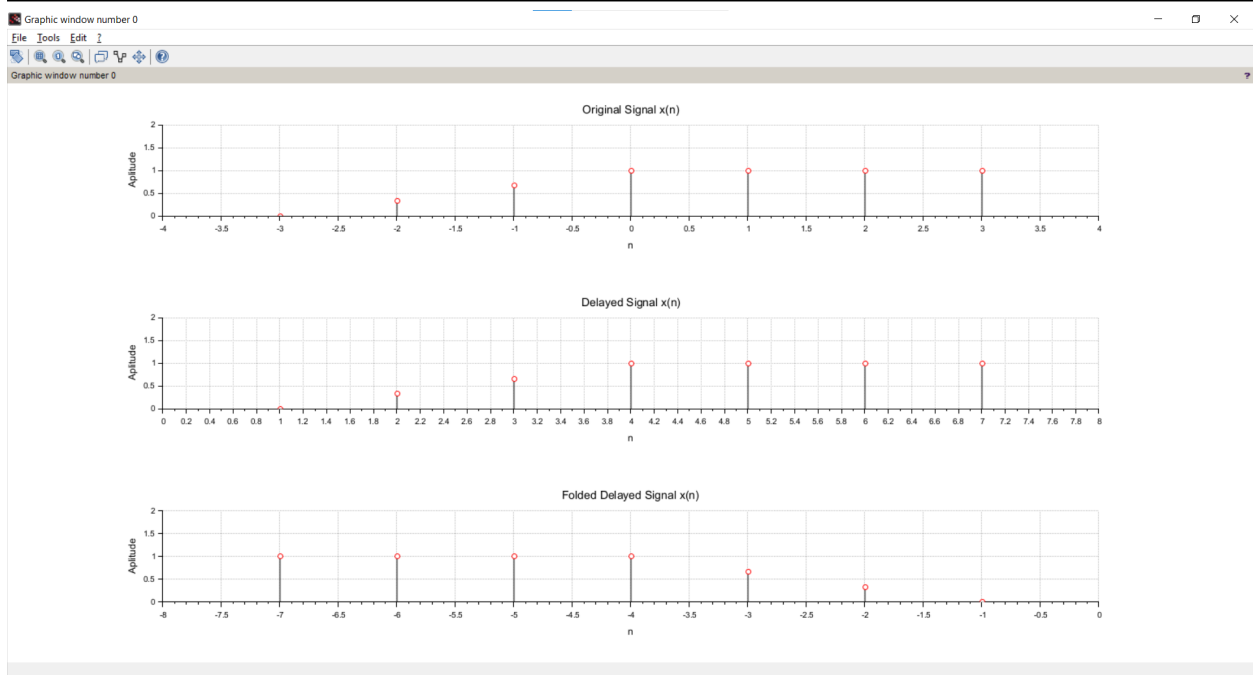


Hình 14: Đồ thị tín hiệu $x(n)$ ban đầu, sau khi lật rồi làm trễ đi 4 mẫu.

2. Đầu tiên làm trễ $x(n)$ đi bốn mẫu và sau đó lật tín hiệu kết quả.

Đoạn code Scilab làm trễ $x(n)$ đi bốn mẫu, tiếp tục lật tín hiệu rồi vẽ tín hiệu kết quả.

```
1 //----- Exercise 2.1b2 -----
2 disp(xn);
3 subplot(3, 1, 1);
4 draw(xn, xorigin);
5 title("Original Signal x(n)");
6
7 [xn_delay, xorigin_delay] = delay(xn, xorigin, 4);
8 disp(xn_delay);
9 subplot(3, 1, 2);
10 draw(xn_delay, xorigin_delay);
11 title("Delayed Signal x(n)");
12
13 [xnb2, xnb2origin] = fold(xn_delay, xorigin_delay);
14 disp(xnb2);
15 subplot(3, 1, 3);
16 draw(xnb2, xnb2origin);
17 title("Folded Delayed Signal x(n)");
18
```



Hình 15: Đồ thị tín hiệu $x(n)$ ban đầu, bị làm trễ đi 4 tín hiệu rồi lật tín hiệu.

- (c) Vẽ đồ thị tín hiệu $x(-n + 4)$. Đồ thị cho tín hiệu $x(-n + 4)$ chính là đồ thị kết quả ở câu b1 (hình 14).
- (d) So sánh các kết quả ở phần (b) và (c), từ đó rút ra quy tắc để thu được tín hiệu $x(-n + k)$ từ $x(n)$.

Dựa trên kết quả từ câu (b) và câu (c), ta có các nhận xét sau:

- Tín hiệu thu được ở câu (b.1) (gấp trước, dịch sau) hoàn toàn trùng khớp với biểu thức $x(-n + 4)$ ở câu (c).
- Tín hiệu ở câu (b.2) (dịch trước, gấp sau) cho kết quả là $x(-n - 4)$, khác với câu (c).

Quy tắc tổng quát: Để thu được tín hiệu $x(-n + k)$ từ tín hiệu gốc $x(n)$, ta thực hiện theo trình tự:

- (a) **Phép gấp (Folding):** Lấy đối xứng $x(n)$ qua trục tung để được $x(-n)$.
- (b) **Phép dịch (Shifting):**

- Nếu $k > 0$: Dịch tín hiệu đã gấp sang **phải** k đơn vị.
- Nếu $k < 0$: Dịch tín hiệu đã gấp sang **trái** $|k|$ đơn vị.

- (e) Bạn có thể biểu diễn tín hiệu $x(n]$ theo các tín hiệu xung đơn vị $\delta(n]$ và bước đơn vị $u(n]$ không?

Trước hết, ta xác định các giá trị cụ thể của $x(n]$ dựa trên hàm phân đoạn:

n	-3	-2	-1	0	1	2	3
$x(n]$	0	1/3	2/3	1	1	1	1

1. Biểu diễn qua dãy xung đơn vị $\delta(n]$

Sử dụng định nghĩa $x(n] = \sum_{k=-\infty}^{\infty} x(k)\delta(n - k]$, ta có:

$$x(n] = \frac{1}{3}\delta(n + 2) + \frac{2}{3}\delta(n + 1) + \delta(n) + \delta(n - 1) + \delta(n - 2) + \delta(n - 3) \quad (2)$$

2. Biểu diễn qua hàm bước đơn vị $u(n]$

Vì $x(n]$ là tín hiệu rời rạc, ta có thể biểu diễn các giá trị tăng dần của nó bằng cách cộng dồn các hàm nấc đơn vị $u(n]$ tại các thời điểm tương ứng. Biểu thức của tín hiệu là:

$$x(n] = \frac{1}{3}u(n + 2) + \frac{1}{3}u(n + 1) + \frac{1}{3}u(n) - u(n - 4) \quad (3)$$

Giải thích:

- Tại $n = -2$, hàm $\frac{1}{3}u(n + 2)$ đưa biên độ lên 1/3.
- Tại $n = -1$, hàm $\frac{1}{3}u(n + 1)$ cộng thêm 1/3, đưa biên độ lên 2/3.
- Tại $n = 0$, hàm $\frac{1}{3}u(n)$ cộng thêm 1/3 cuối cùng, đưa biên độ lên 1 và duy trì hằng số.
- Tại $n = 4$, hàm $-u(n - 4)$ triệt tiêu toàn bộ biên độ, đưa tín hiệu về 0.

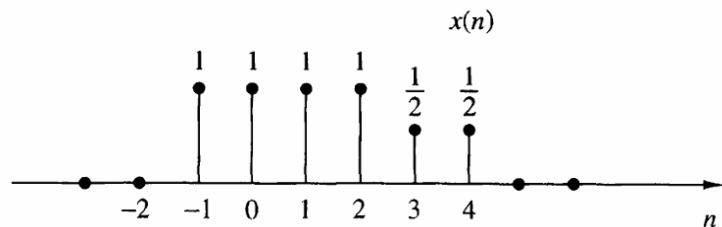
4.8 Exercise 2.2

Tín hiệu thời gian rời rạc $x(n]$ được cho như ở Hình P2.2. Hãy vẽ và ghi chú cẩn thận từng tín hiệu sau đây:

(a) $x(n - 2)$

- (b) $x(4 - n)$
- (c) $x(n + 2)$
- (d) $x(n)u(2 - n)$
- (e) $x(n - 1)\delta(n - 3)$
- (f) $x(n^2)$
- (g) Thành phần chẵn của $x(n)$ (even part)
- (h) Thành phần lẻ của $x(n)$ (odd part)

Figure P2.2



Hình 16: Hình ảnh cho Exercise 2.2

Bài làm

Đoạn code Scilab làm trẽ $x(n)$ đi bốn mẫu, tiếp tục lật tín hiệu rồi vẽ tín hiệu kết quả.

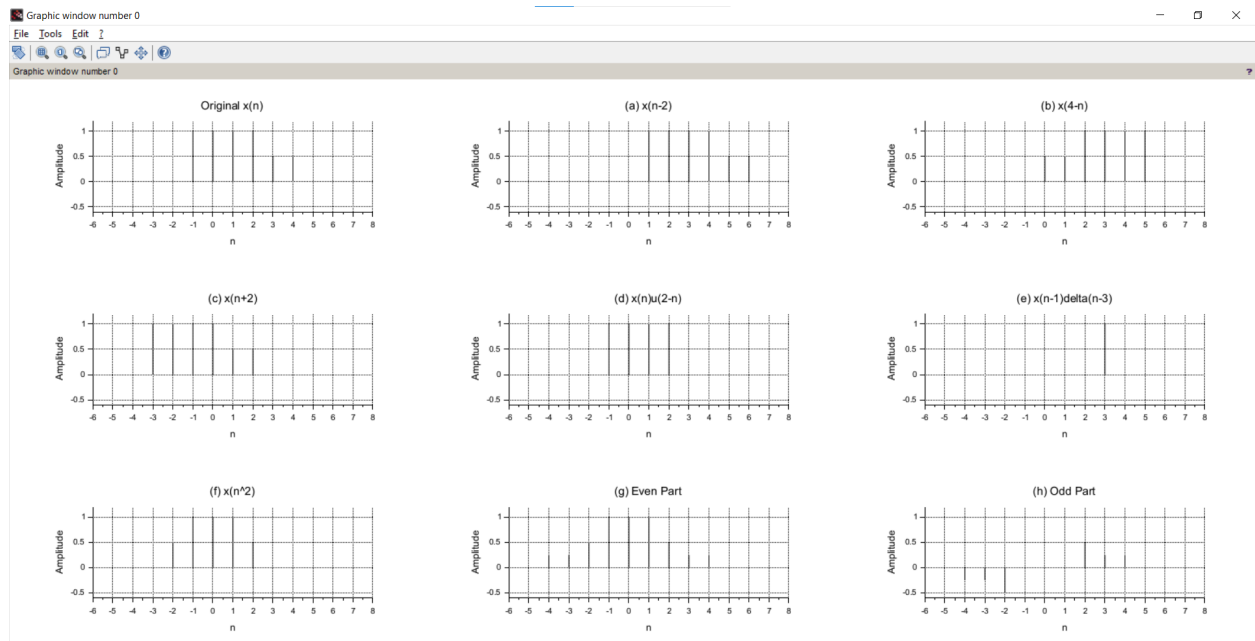
```
1 clear; clc;
2
3 // 1. DEFINE TIME RANGE AND ORIGINAL SIGNAL X(N)
4 // Define a wide range for 'n' to cover all shifts and folding
5 n = -10:10;
6
7 // Function to return x(k) based on Fig P2.2 definition
8 function val = get_x(k)
9     // x(n) = 1 for n = -1, 0, 1, 2
10    if (k >= -1 & k <= 2) then
11        val = 1;
12    // x(n) = 0.5 for n = 3, 4
13    elseif (k == 3 | k == 4) then
14        val = 0.5;
```

```
15 // x(n) = 0 elsewhere
16 else
17     val = 0;
18 end
19 endfunction
20
21 // Initialize arrays for all transformed signals
22 xn = zeros(1, length(n)); // Original x(n)
23 xa = zeros(1, length(n)); // (a) x(n-2)
24 xb = zeros(1, length(n)); // (b) x(4-n)
25 xc = zeros(1, length(n)); // (c) x(n+2)
26 xd = zeros(1, length(n)); // (d) x(n)u(2-n)
27 xe = zeros(1, length(n)); // (e) x(n-1)delta(n-3)
28 xf = zeros(1, length(n)); // (f) x(n^2)
29 xg = zeros(1, length(n)); // (g) Even part
30 xh = zeros(1, length(n)); // (h) Odd part
31
32 // 2. CALCULATE VALUES FOR EACH CASE
33 for i = 1:length(n)
34     curr_n = n(i);
35     xn(i) = get_x(curr_n);
36
37     // (a) Delay by 2 samples
38     xa(i) = get_x(curr_n - 2);
39
40     // (b) Fold and then shift right by 4 (x(4-n))
41     xb(i) = get_x(4 - curr_n);
42
43     // (c) Advance by 2 samples
44     xc(i) = get_x(curr_n + 2);
45
46     // (d) Multiplication by unit step u(2-n)
47     if (curr_n <= 2) then xd(i) = get_x(curr_n); else xd(i) = 0; end
48
49     // (e) Multiplication by unit impulse delta(n-3)
50     if (curr_n == 3) then xe(i) = get_x(curr_n - 1); else xe(i) = 0; end
51
52     // (f) Non-linear time scaling
```

```

53     xf(i) = get_x(curr_n^2);
54
55     // (g) Even component formula: 0.5 * [x(n) + x(-n)]
56     xg(i) = 0.5 * (get_x(curr_n) + get_x(-curr_n));
57
58     // (h) Odd component formula: 0.5 * [x(n) - x(-n)]
59     xh(i) = 0.5 * (get_x(curr_n) - get_x(-curr_n));
60 end
61
62 // 3. PLOTTING THE RESULTS
63 clf(); // Clear current figure
64 scf(0); // Set current figure to ID 0
65
66 // We zoom into n = [-5, 8] for better visibility
67 n_zoom = -5:8;
68 idx = (n_zoom - n(1)) + 1; // Indices in the main 'n' array
69
70 // Define a helper function to set plot properties and prevent the Min/
    Max error
71 function format_plot(title_str)
72     xtitle(title_str, "n", "Amplitude");
73     a = gca();
74     // Correct order: [xmin, ymin; xmax, ymax]
75     a.data_bounds = [-5, -0.6; 8, 1.2];
76     a.grid = [1, 1]; // Enable grid for easier reading
77 endfunction
78
79 // Plot all 9 signals in a 3x3 grid
80 subplot(3,3,1); plot2d3(n_zoom, xn(idx)); format_plot("Original x(n)");
81 subplot(3,3,2); plot2d3(n_zoom, xa(idx)); format_plot("(a) x(n-2)");
82 subplot(3,3,3); plot2d3(n_zoom, xb(idx)); format_plot("(b) x(4-n)");
83 subplot(3,3,4); plot2d3(n_zoom, xc(idx)); format_plot("(c) x(n+2)");
84 subplot(3,3,5); plot2d3(n_zoom, xd(idx)); format_plot("(d) x(n)u(2-n)");
85 subplot(3,3,6); plot2d3(n_zoom, xe(idx)); format_plot("(e) x(n-1)delta(n
    -3)");
86 subplot(3,3,7); plot2d3(n_zoom, xf(idx)); format_plot("(f) x(n^2)");
87 subplot(3,3,8); plot2d3(n_zoom, xg(idx)); format_plot("(g) Even Part");
88 subplot(3,3,9); plot2d3(n_zoom, xh(idx)); format_plot("(h) Odd Part");

```

Hình 17: Đồ thị các tín hiệu theo yêu cầu Exercise 2.2

5 Bàn Luận

5.1 Kết quả đạt được

Thông qua bài thực hành Lab 3, em đã hoàn thành các nội dung trọng tâm sau:

- **Về mặt lý thuyết:** Đã nắm vững và phân biệt được sự khác biệt giữa các phép biến đổi thời gian. Đặc biệt là quy tắc "Gấp trước - Dịch sau" để xác định chính xác biểu thức $x(-n + k)$.
- **Về mặt tính toán:** Đã thực hiện thành công việc tách tín hiệu thành các thành phần đối xứng (chẵn/lẻ) và biểu diễn tín hiệu qua các hàm cơ bản $\delta(n)$ và $u(n)$.
- **Về mặt công cụ:** Sử dụng thành thạo phần mềm Scilab để mô phỏng tín hiệu. Việc lập trình hàm `get_x(k)` giúp giải quyết triệt để vấn đề lệch chỉ số mảng (index) giữa toán học và lập trình.

5.2 Những tồn tại và hạn chế

Mặc dù kết quả mô phỏng khớp với lý thuyết, tuy nhiên vẫn còn một số điểm cần lưu ý:

- Trong quá trình làm bài 2.1, ban đầu còn có sự nhầm lẫn về gốc tọa độ và thứ tự thực hiện phép trễ sau khi gấp. Tuy nhiên, sau khi kiểm chứng bằng hình vẽ và biểu thức toán học, lỗi này đã được khắc phục.
- Việc xử lý các phép biến đổi thang thời gian phi tuyến như $x(n^2)$ trong bài 2.2 đòi hỏi sự cẩn trọng cao trong việc lấy mẫu để không làm mất các đặc điểm quan trọng của tín hiệu.

5.3 Định hướng phát triển kỹ năng

Trong thời gian tới, để nâng cao năng lực xử lý tín hiệu số, em định hướng:

- Tiếp tục làm chủ các thư viện đồ họa nâng cao trong Scilab để trực quan hóa tín hiệu tốt hơn.
- Tìm hiểu sâu hơn về ứng dụng của các phép biến đổi này trong các hệ thống thực tế như lọc nhiễu tín hiệu hoặc nén dữ liệu.
- Rèn luyện kỹ năng trình bày báo cáo kỹ thuật bằng $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ để đảm bảo tính chuyên nghiệp và chính xác của các công thức toán học phức tạp.

6 Tài liệu tham khảo

Source code Scilab cho các bài tập Lab 3: https://github.com/NguyenNgocQuynhNhi/HK252_DigitalSignalProcessing_Lab
https://www.scilab.org/sites/default/files/Scilab_beginners.pdf
https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes
https://www.overleaf.com/learn/latex/List_of_Greek_letters_and_math_symbols